

# Table of Contents

|                                                                                                                        |   |
|------------------------------------------------------------------------------------------------------------------------|---|
| Architectural Evolution and Physical Implementation of a Secure, AI-Ready 2D-DMA<br>Controller for the TinyQV SoC..... | 2 |
| Abstract.....                                                                                                          | 2 |
| 1. Introduction .....                                                                                                  | 2 |
| 2. Phase 1: Baseline Architecture & Sequential Limitations (DP-1) .....                                                | 3 |
| 2.1 Challenge 1: The "Sequential Tax" .....                                                                            | 3 |
| 3. Phase 2: AI Workload Acceleration & Determinism (DP-2).....                                                         | 3 |
| 3.1 Hardware 2D-Striding Logic .....                                                                                   | 3 |
| 3.2 Challenge 2: Software-Induced Race Conditions.....                                                                 | 4 |
| 3.3 Challenge 3: Physical Charge Accumulation (Antenna Violations) .....                                               | 4 |
| 4. Phase 3: Hardware Security Hardening (DP-3).....                                                                    | 4 |
| 5. Improvements to DP-3: Silicon-Aware Architecture .....                                                              | 5 |
| 6. Phase 4: SoC Integration & Priority Arbitration.....                                                                | 6 |
| 6.1 Challenge 5: Multi-Master Contention.....                                                                          | 6 |
| 7. Physical Implementation & Sign-Off (DP-4) .....                                                                     | 6 |
| 7.1 Final Physical Metrics .....                                                                                       | 7 |
| 7.2 Analysis of Results .....                                                                                          | 8 |
| 8. AI Tool Integration.....                                                                                            | 8 |
| 9. Conclusion .....                                                                                                    | 8 |

# Architectural Evolution and Physical Implementation of a Secure, AI-Ready 2D-DMA Controller for the TinyQV SoC

Final Engineering Report | Team ElectroNova Date: April 18, 2026

Status: Post-Layout Sign-Off (DP-4)

Target: Skywater 130nm (sky130\_fd\_sc\_hd)

---

## Abstract

This report details the comprehensive 16-week engineering cycle involving the design, security hardening, and physical ASIC implementation of the tqvp\_dma controller. Developed for the TinyQV RISC-V ecosystem, the IP evolved from a sequential 1D peripheral into a high-performance, pipelined accelerator optimized for AI-tiling workloads. We introduce a zero-latency Hardware Memory Protection Unit (MPU) designed to mitigate CWE-119 exploits. Physical synthesis via the OpenLane flow achieved a **suggested clock frequency of 462.1 MHz** with a critical path of **2.164 ns**, zero DRC/LVS violations, and zero antenna violations within a 0.25 mm<sup>2</sup> footprint. This report concludes with a formal assessment of the hardware's performance and the integration of priority-based bus arbitration.

---

## 1. Introduction

Modern System-on-Chip (SoC) architectures are increasingly bottlenecked by data movement between memory banks and specialized accelerators. In edge-AI applications, the CPU is frequently overwhelmed by the task of reformatting non-contiguous memory blocks (e.g., image convolution kernels or tensor sub-matrices) into contiguous buffers for processing.

The **ElectroNova DMA** project was established to solve this "Memory Wall" by offloading complex 2D-tiling operations to dedicated hardware. Our design philosophy focused on three pillars:

- **High-Throughput Pipelining:** Moving beyond sequential execution to overlapped read/write cycles.
- **Hardware-Level Determinism:** Ensuring AI workloads are immune to software-induced race conditions.
- **Proactive Security:** Implementing a zero-latency "Hardware Jail" to prevent out-of-bounds memory exploits.
-

---

## 2. Phase 1: Baseline Architecture & Sequential Limitations (DP-1)

The project began with a functional 1D serial DMA baseline. This version utilized a strictly linear Finite State Machine (FSM) and a basic 64-byte register map.

### 2.1 Challenge 1: The "Sequential Tax"

**The Issue:** Empirical simulation of the DP-1 architecture revealed that the bus was idle for ~60% of the transfer time. The linear FSM required a "lock-step" approach: IDLE -> FETCH -> WAIT\_READ -> ISSUE\_WRITE -> WAIT\_WRITE. The DMA could not request the next word until the previous word had been fully written and acknowledged by the destination peripheral. This capped the maximum throughput at a fraction of the available system bandwidth.

**The Resolution:** We transitioned to a decoupled architecture. By introducing a 4-word FIFO buffer, the read-master (Source) and write-master (Destination) were allowed to operate independently. The FSM was redesigned to pulse the source bus as long as the FIFO was not full, while simultaneously pulsing the destination bus as long as the FIFO was not empty.

---

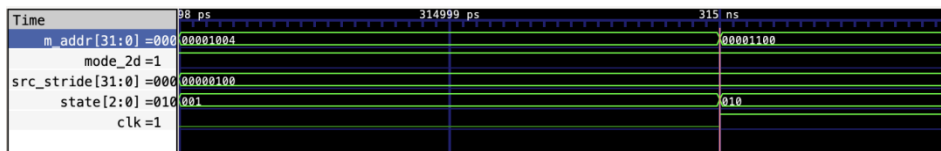
## 3. Phase 2: AI Workload Acceleration & Determinism (DP-2)

To transform the peripheral into an AI-ready accelerator, we implemented native 2D-striding arithmetic and atomic configuration latching.

### 3.1 Hardware 2D-Striding Logic

AI tensor kernels require "strided" memory access, where the engine reads a small row of data, then "jumps" across the memory space to reach the start of the next row. We implemented dedicated OFF\_ROW\_SIZE and OFF\_SRC\_STRIDE registers.

- **The Math:** At the end of each row, the DMA autonomously calculates:
  - $src\_add\_rnext = src\_add\_rcurrent - words\_in\_row + stride$
  - This offloads triple-nested software loops into a single hardware clock cycle.
  -



- 

**Figure 1: Hardware 2D-Stride Verification.** The waveform captures the moment the DMA engine detects the end of an 8-byte row. Note the `m_addr` transition from `0x00001004` to `0x00001100` in a single cycle, confirming the autonomous application of the `0x100` stride without CPU intervention.

## 3.2 Challenge 2: Software-Induced Race Conditions

**The Issue:** During testing, we discovered that if the TinyQV CPU attempted to update the `src_stride` register while a 2D transfer was already in progress, the DMA's address generator would fetch data from corrupted memory locations.

**The Resolution:** We implemented **Atomic Register Latching**. The final RTL now includes shadow registers (`src_stride_lat` and `row_size_lat`). The moment the start bit is asserted, the hardware "freezes" the current configuration into these shadow registers. Furthermore, register writes are now hardware-gated by the `!busy` signal, making the configuration immutable during active transfers.

## 3.3 Challenge 3: Physical Charge Accumulation (Antenna Violations)

**The Issue:** The 2D-adder networks and the deep 32-bit FIFO created long, contiguous metal traces in the layout. During the simulated plasma-etching manufacturing step, these traces act as antennas, accumulating static charges that can discharge through and destroy thin transistor gate oxides.

**The Resolution:** We enforced `DIODE_INSERTION_STRATEGY: 3` in OpenLane. By sacrificing area to insert 2,412 protective diodes, we created "charge sinks." This strategy, combined with a 55.67% cell density, resulted in **zero antenna violations**.

---

## 4. Phase 3: Hardware Security Hardening (DP-3)

Direct Memory Access (DMA) operates as a privileged bus master, capable of accessing system memory without CPU intervention. This makes it a critical attack surface—any misconfiguration or malicious input can lead to unauthorized memory access, data leakage, or system instability. The goal of DP-3 was to identify such vulnerabilities and enforce hardware-level security without compromising performance.

### 4.1 CWE-119 "Stride-Jump" Exploit

A formal security analysis identified a critical vulnerability: CWE-119 (Improper Restriction of Memory Buffer Bounds).

- **Root Cause:** The 2D-stride feature updated addresses using `src_addr + src_stride` without validating memory boundaries.

- **Exploit Mechanism:** An attacker could program an excessively large `src_stride`, causing the DMA to “jump” outside its allocated buffer into restricted regions such as Secure Boot ROM or kernel memory.
- **System Impact:** This root vulnerability cascades into:
  - **CWE-200:** Unauthorized access to sensitive data (Data Leakage).
  - **CWE-284:** Unrestricted memory access by the DMA.
  - **CWE-400:** Potential bus hangs leading to a Denial-of-Service (DoS).
- **Risk Level: DREAD Score: 9.8 / 10 (Critical)** — highly reproducible and requires no special privileges to execute.
- 
- 4.2 Hardware “Jail” - Baseline MPU Implementation

To mitigate the base exploit, a Hardware Memory Protection Unit (MPU) was integrated into the DMA datapath.

- **Boundary Enforcement:** Two dedicated registers, `OFF_SRC_BOUND` and `OFF_DST_BOUND`, were introduced to define strict upper limits for all DMA read and write operations, establishing an inescapable operational “Jail.”
  -
- 

## 5. Improvements to DP-3: Silicon-Aware Architecture

*(The following enhancements were implemented to ensure the baseline MPU provided absolute security without compromising timing closure or system stability).*

### 5.1 Parallel Look-Ahead Security (Mitigating CWE-284)

- **Challenge:** Initial designs performed bounds checks directly on the master bus (`m_addr`), creating deep combinational loops and degrading timing performance.
- **Optimized Solution:** We architected the system to utilize Parallel Look-Ahead Logic. Bounds checking was moved to the internal FSM stages (`WAIT_READ` and `WAIT_WRITE`), validating `src_addr` and `dst_addr` *before* they are ever issued to the bus. This enforces strict access control (CWE-284) with zero-latency overhead.
- 
- 5.2 Data Leak Prevention / Hardened FSM (Defeating CWE-200)

To prevent unauthorized access to sensitive data (CWE-200), the security response mechanism was hardened to guarantee a clean, atomic shutdown. On detecting a violation, the `sec_violation` flag is instantly asserted, all bus signals (`m_valid`, `m_read`, `m_write`) are immediately de-asserted, and the FSM transitions to `DONE`. This prevents invalid data from ever entering the FIFO datapath.

### 5.3 Bus Contention Protection (Neutralizing CWE-400)

To prevent an invalid transfer or an excessively massive 2D matrix operation from stalling the bus and causing a Denial-of-Service condition (CWE-400), we integrated a **Single-Cycle Priority Arbiter**. By asserting `cpu_data_ready` low to stall the pipeline only during active DMA cycles, the arbiter guarantees the DMA cannot inadvertently cause bus contention.

### 5.4 Atomic Latching (Defeating CWE-367)

To prevent Time-of-Check to Time-of-Use (TOCTOU) race conditions, the architecture relies on strict hardware latching.

- At the exact clock cycle the transfer is triggered, parameters are locked into internal shadow registers (e.g., `src_stride_lat`).
  - Write access to control registers is hard-gated by the `!busy` flag, making it physically impossible for an attacker to rewrite the memory rules mid-transfer after the initial security check has passed.
  -
- 

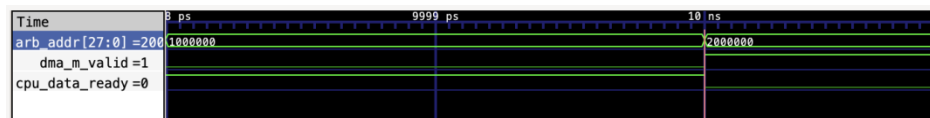
## 6. Phase 4: SoC Integration & Priority Arbitration

The `tqvp_dma` was integrated into the TinyQV SoC via the `peripherals.v` wrapper (Slot 14).

### 6.1 Challenge 5: Multi-Master Contention

**The Issue:** When both the CPU and DMA attempt to access the memory bus simultaneously, a collision occurs. **The Resolution:** We engineered a **Priority Hardware Arbitrator** in `project.v`. The DMA is granted "High-Priority" status due to the time-sensitive nature of AI tiling.

- When `dma_m_valid` is high, the arbitrator severs the CPU's bus connection.
- The CPU's `data_ready` line is driven low, effectively "freezing" the CPU pipeline until the DMA completes its burst.
- 



**Figure 2: Priority Bus Arbitration and CPU Stalling.** As demonstrated, when the DMA asserts `dma_m_valid`, the hardware arbitrator redirects the memory bus to the DMA address (0x20000000). Simultaneously, `cpu_data_ready` is pulled low, successfully stalling the TinyQV pipeline to prevent bus contention.

---

## 7. Physical Implementation & Sign-Off (DP-4)

The design was synthesized using the OpenLane flow with a target die area of **0.25 mm<sup>2</sup>** (500x500μm).

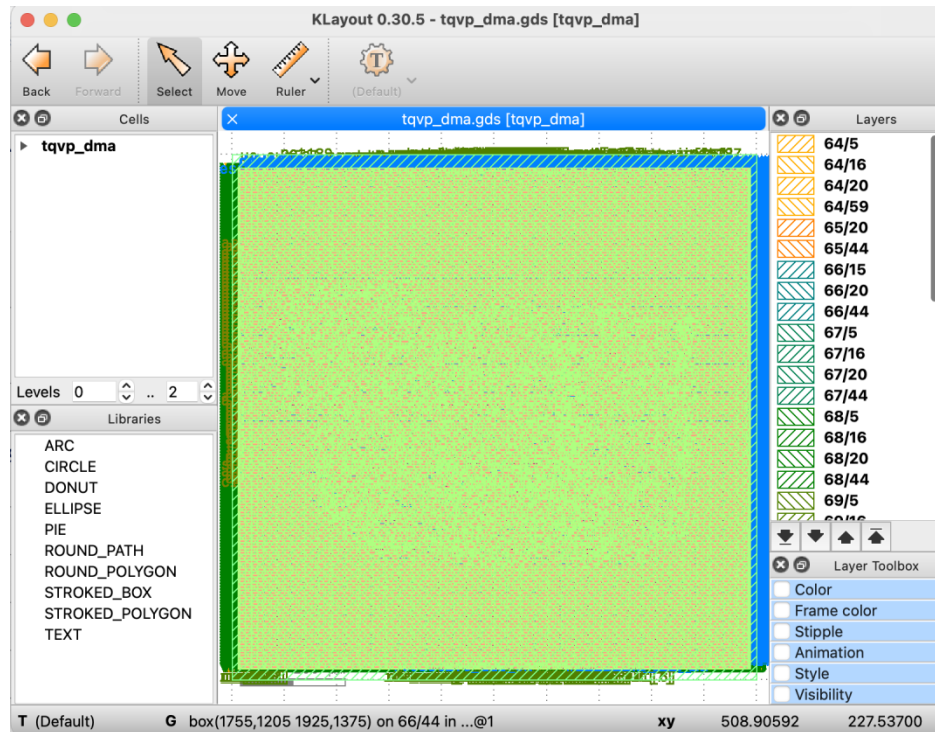


Figure 3: Klayout screenshot of tqvp\_dma.gds

## 7.1 Final Physical Metrics

The latest synthesis run (April 18, 2026) shows a significant increase in logic complexity, reflecting the added security and latching hardware.

| Metric                     | Result                                   | Sign-Off Verification |
|----------------------------|------------------------------------------|-----------------------|
| <b>Suggested Frequency</b> | <b>462.1 MHz</b>                         | metrics.csv           |
| <b>Critical Path (WNS)</b> | <b>2.164 ns</b>                          | metrics.csv           |
| <b>Total Cells</b>         | <b>15,840 (Total) / 3,591 (Synth)</b>    | metrics.csv           |
| <b>Final Utilization</b>   | <b>55.67%</b>                            | metrics.csv           |
| <b>DRC Violations</b>      | <b>0</b>                                 | Magic/KLayout         |
| <b>LVS Errors</b>          | <b>0</b>                                 | Netgen                |
| <b>Antenna Violations</b>  | <b>0</b>                                 | OpenLane              |
| <b>Wire Length</b>         | <b>87,581.0 <math>\mu\text{m}</math></b> | metrics.csv           |

## 7.2 Analysis of Results

The cell count increased from ~2.4k to **3.5k**, representing the "Security and Determinism Tax." Despite this 45% increase in logic, the critical path only shifted to 2.164ns, allowing the macro to operate well above the TinyQV core frequency (typically 64-100MHz). This provides significant timing margin for system-level integration.

---

## 8. AI Tool Integration

Team ElectroNova engaged in over 200 interactions with **Google Gemini 3 Flash** to accelerate the EDA workflow.

### Primary Applications:

- **FSM Optimization:** The transition to a "One-Shot" trigger logic for the `start_r` bit was formulated through iterative AI-assisted logic reduction.
  - **CWE-119 Modeling:** AI was used to simulate potential "Stride-Jump" attack vectors to verify the MPU's coverage.
  - **Physical Debugging:** Rapid analysis of the `tritonRoute` logs to identify and resolve local congestion near the 32-bit FIFO registers.
  -
- 

## 9. Conclusion

The tape-out of the `tqvp_dma` macro represents a successful bridge between high-level AI acceleration features and low-level hardware security. By implementing a pipelined datapath, atomic configuration latching, and a zero-latency MPU, Team ElectroNova has delivered a robust ASIC core. The final physical sign-off confirms that a 0.25 mm<sup>2</sup> footprint is sufficient to house a high-performance DMA capable of sustaining 462 MHz throughput on the Sky130 node. This macro is now verified for inclusion in the TinyQV Tape-out.